

Wallet QA agent developer handoff

Prepared on 6 October 2026, Asia Singapore. This is a product and engineering brief for a developer team taking over the selected TOKEN2049 Origins idea. It contains planning documents and illustrative records, with no application implementation or completed test runs.

Build a paid Web3 QA agent that agrees on expected behavior with a buyer, uses a real isolated browser wallet, executes an approved test plan, and delivers recordings plus a structured account of the session. Cardano and Masumi handle service payment; the tested app can use a different chain.

Read in this order

Document	Purpose
01 Product and scope	Product promise, MVP boundaries, agreed decisions and proposed defaults
02 User journey and test specification	Concise interview, requirements, test cases and report states
03 Execution and budgets	Pricing, contingency, wallet controls, retries and checkpoints
04 Session evidence and refund review	Session logging, fault attribution, human review and Masumi checklist
05 Architecture and implementation sequence	Components, integration boundaries and dependencies
06 Swap demo and acceptance	Manageable full swap demo and delivery checks

The examples directory contains an approved-plan example, a structured session log, and a report example. All are synthetic illustrations of a missing notification after a successful testnet swap. IDs and artifact paths refer to fictional records; hashes and real addresses are deliberately unset.

Wallet QA Handoff.pdf combines the documents for easy reading. The Markdown files remain the editable source; the JSON examples are supplied separately.

Decision status

Agreed: structured interviews and test cases; task-relevant clarification; real wallet interaction; separate testing funds; approved plan and price before execution; escrow before execution; buffered costs; bounded recovery; recordings and step logs; human refund review; a complete swap demo. Agent-to-agent mode is paused until the main flow works. Telegram is optional and lower priority.

Proposed defaults: MetaMask on Base Sepolia for testing, Cardano preprod for payment, approximately 25 percent contingency, three attempts total per eligible case, two included clarification rounds, and full-job refund or replacement run for the MVP. Validate these defaults before implementing production behavior.

Must resolve early: exact demo app and pool, runner and extension compatibility, evidence capture across wallet windows, Masumi hosting and payment asset, deadline configuration, and the responsible human reviewer.

Hackathon constraints

The supplied TOKEN2049 Origins Hackathon Singapore PDF specifies a 36-hour build, meaningful partner integration, and projects built during the event. Existing public tools and libraries are permitted. Execution and technical integration together account for 55 percent of main-track scoring.

Submit a repository, hosted demo, and a Google Drive link to a PPT or Keynote file. Final-stage demos must use recordings embedded in the slides; external video links and live demos are not accepted. The supplied deadline is 11:59 pm on 7 October 2026; confirm the organizer's timezone before submission. Display the project clock in Asia Singapore for this event.

The relevant partner track is Cardano Agentic Commerce. Testing an EVM app does not by itself qualify for the Solana track. Additional partner-track conditions should be checked with organizers.

Wallet QA agent product and scope

The buyer hires a QA agent to investigate a deployed Web3 app and execute a confirmed testing task. The service turns a short request into test cases with explicit expectations, then returns evidence the buyer can inspect and a record a human can use to review delivery disputes.

Buyer and product promise

The initial buyer is a small Web3 team without dedicated end-to-end QA. Its request might be: "Test the complete token swap flow, including wallet approval and success notification."

The purchased deliverable is an evidence-backed test run against agreed requirements. A reproducible app defect is useful delivery. An inability to execute the agreed work is a delivery issue that can be reviewed for a refund or replacement run.

The agent needs a browser wallet with its own isolated test identity. It can connect, approve a bounded amount, sign permitted requests, and execute approved test transactions. The service wallet on Cardano is a separate identity used for Masumi settlement.

MVP scope

- One desktop browser and one supported wallet.
- One deployed app, one test network, and one transaction journey per job.
- A bounded initial inspection followed by a structured interview.
- A reviewable test plan, fixed quote, execution limits and refund deadline.
- Real wallet connection and testnet transactions.
- Recorded browser and wallet interactions, step logs, traces and chain evidence.
- A report distinguishing defects from blockers and automation problems.
- Human review for contested refunds, integrated with Masumi's settlement lifecycle.

The proposed first target is a small EVM staging swap app on Base Sepolia with MetaMask and two test tokens. Cardano preprod pays for the QA service. The user has selected a full swap demo but has not confirmed the exact wallet, chain, app or provider.

Testing modes

Mode	Behavior
Quick check	Apply a standard wallet-onboarding checklist to a confirmed scope
User-defined cases	Convert supplied cases into executable assertions and required evidence
Explore and propose	Discover accessible features, clarify intent and propose prioritized coverage

A budget is a constraint across every mode. Exploration produces a coverage map of tested, discovered, blocked and unexplored features; it does not establish exhaustive coverage of an arbitrary website.

Core product decisions

The buyer approves expected behavior rather than asking the model to infer correctness entirely from the app. The plan is versioned and preserved during execution. A change in scope, assertions or spending requires an explicit revision.

The quote contains a bounded allowance for hidden clarification and recovery costs. Recovery stops after its approved limit. Cases that remain unresolved are visible in the report and eligible for human delivery review.

Every job-relevant interaction and execution action is captured in a structured session record. Browser and wallet actions also have recordings where applicable. Fault attribution distinguishes observed evidence, proposed cause and the human decision; unknown responsibility remains unknown.

Deferred work

Agent-to-agent hiring is paused until the human main flow works. Preserve structured input and status boundaries so a later agent client can use the same backend. Do not build external agent discovery, payment delegation or multi-agent negotiations in the MVP.

Telegram is an optional later communication channel over the same job record. Interviews, progress and report delivery can happen in chat; authenticated plan approval and funding can use a web or Mini App flow.

GitHub read access, recurring runs, additional wallets, mobile wallet flows, multiple networks, partial escrow refunds, and unattended mainnet execution are later work. A URL and confirmed acceptance criteria are sufficient for the first demo.

Definition of useful delivery

A buyer can see what was requested, what was attempted, what happened, which evidence supports each result, and what remains unresolved. Another developer can reproduce an app finding from the recorded conditions and steps. A human reviewer can assess whether promised testing and evidence were delivered.

Wallet QA agent user journey and test specification

The workflow follows a compact testing lifecycle: user goal, confirmed requirements, test design, approved execution, evidence, findings and optional retest. The interview asks only for information that changes the task's assertions, prerequisites or permitted actions.

Main user journey

1. The buyer supplies the URL, testing goal, budget ceiling and whether transactions are permitted.
2. The agent inspects a bounded set of pages and identifies visible flows, likely wallet/network requirements and missing context.
3. The UI presents the agent's understanding with source labels and a short batch of essential questions.
4. The buyer corrects assumptions, confirms intended behavior and supplies prerequisites. Funding requests specify the address, network, asset and amount.
5. The agent proposes prioritized cases, expected outcomes, required proof, fixed price, deadlines and execution limits.
6. The buyer approves the exact plan version and quote. Funds lock in Masumi escrow before paid execution.
7. The runner executes the plan. Unexpected prerequisites produce a checkpoint; independent cases may continue within the approved scope.
8. The buyer receives the report and evidence, then accepts delivery or requests human review within the settlement deadline.

UI completion and settlement completion are separate. A report can be ready while a refund request or payment collection is still pending.

Context and essential questions

Context source	Use
Deployed URL	Visible behavior, navigation, console and network observations
Product requirements or walkthrough	Intended behavior and meaningful assertions
Optional repository read access	Relevant routes, existing tests, contract configuration and deployment mapping
Test fixtures	Allowlisted accounts, required balances, seeded data and repeatable conditions

Store the provenance of each context field as user-confirmed, documented, observed or assumed. Code and current UI behavior are inputs; either can contain the bug being tested.

Question fields: id, field, why_needed, options, required, affected_case_ids, answer, source. Ask before execution when missing input changes correctness or spending. Bundle a few essential questions and use discovered values as editable defaults. Optional preferences can use disclosed defaults.

For a swap, confirm the network and wallet; input/output token identities and decimals; allowed router/spender; input amount and minimum output or slippage policy; expected notification behavior and timeout; required account/data/funding; and the target deployment.

Shared job specification

Store environment and limits once per plan: URL, build identifier, chain ID, wallet, contracts, asset metadata, fixture references, budgets, recording requirements and deadlines. Test cases reference those fields by ID.

The approved plan includes its version, approval identity/time, exact input snapshot, case list, scope exclusions and quote. Freeze the snapshot once funded. Relevant environment or expectation changes invalidate affected cases and require a recorded revision; a funded job does not silently become a larger job.

Test case specification

Field	Required content
Case ID and requirement ID	Stable traceability between intent and test
Priority	Critical, high or normal
Preconditions and dependencies	Wallet/data state and cases that must succeed first
Steps	Ordered observable actions
Expected results	Specific assertions, including thresholds and timeouts
Evidence	Recording segments, screenshots, trace entries and chain checks
Recovery limits	Maximum attempts and case time budget

Example: SWAP-01 covers the approved swap requirement. Given a funded wallet with zero allowance, connect, approve the exact input amount, and submit one swap. Verify the selected account, successful approval receipt and allowance, successful swap receipt, and output amount at least the approved minimum. A linked NOTIFY-01 case checks the expected UI notification without sending another swap. Gas is accounted separately from ERC20 balance changes.

Use Given/When/Then reasoning, explicit prerequisites, positive and negative cases, risk-based priority, and requirement-to-evidence links. Keep a transaction journey dependent where necessary; isolate negative cases with separate fixtures so they do not corrupt the main run.

Result states

State	Meaning
PASS	Approved assertions met with required evidence
FLAKY	Passed only after a recoverable failure; earlier evidence retained
FAIL	Evidence shows an app mismatch against a confirmed expectation
BLOCKED	A prerequisite prevents the case from running
INCONCLUSIVE	Attempts exhausted without enough evidence to determine the outcome
RUNNER_ERROR	Service automation failed to execute reliably
NOT_RUN	Case omitted because of a dependency, limit or explicit stop

Do not label a case PASS when its required proof is missing. Separate case outcome from the proposed cause and from service delivery completeness. A report containing a defect can be fully delivered; a report containing several unfinished critical cases can be incomplete.

Token economy

- Reuse short templates for wallet connection, approval, rejection and confirmation.
- Persist structured answers and send changed fields rather than whole chat histories.
- Fetch relevant repository files and DOM regions rather than repeatedly scanning everything.
- Generate the test script after approval; use deterministic runner assertions for execution.
- Request model diagnosis only for new evidence or recovery decisions.
- Store large logs and recordings as artifacts; pass excerpts and references to the model.

A later machine client can receive the same questions as an input schema. Masumi's service interface supports structured input and an awaiting-input state; public agent hiring remains deferred. [MIP-003](#)

Wallet QA agent execution and budgets

Execution stays within a confirmed scope, fixed service quote and separate transaction limits. The quote includes bounded recovery work so ordinary clarification does not create surprise charges.

Pricing proposal

Stage	Proposed treatment
Initial inspection	Free, with page, browser-time and model-usage ceilings
Deeper discovery	Later optional fixed-price deliverable; credit toward execution in application pricing
Execution	Fixed scoped fee funded into escrow before the runner starts

Start with approximately 25 percent contingency above the estimated service cost and margin. Include up to two clarification rounds and three total attempts per eligible case. These are proposed defaults, not measured unit economics. Benchmark runner costs and choose actual time limits before accepting paid jobs.

Publish one fixed fee including contingency. An unused allowance does not automatically reduce that fee. A metered price with unused-funds refunds would be a different product and needs separate settlement design. Incomplete promised delivery still permits refund review.

The operator finances browser and model consumption while funds remain locked; escrow protects against nonpayment but does not make the locked fee available for immediate operating costs.

Three separate budgets

1. **Service fee:** payment for agreed testing and evidence, including the contingency.
2. **Testing funds:** native tokens and assets held by the job's isolated test wallet. These may be spent or locked by approved tests; report ending balances and outstanding allowances.
3. **Execution limits:** model usage, browser duration, attempts, permitted transactions and pause deadlines enforced by the backend.

Use asset IDs, chain IDs, decimals and base-unit strings for token limits. Fiat equivalents are informational estimates. No service fee, testing funds or gas budget silently converts into another category.

Service refunds concern the QA fee. Spent gas and funds locked in the tested app are not automatically recovered by a Masumi refund. Agree on the return address and handling of unused test funds before accepting deposits; record any authorized return separately.

Wallet execution

Provision an isolated test wallet for the job. Its key stays in the wallet/runner boundary and outside model prompts, chat messages and evidence artifacts. A separate Cardano identity receives service settlement.

Before a signature, enforce approved chain, domain, contract/spender, action type, token amount, gas cap and transaction count in code. Keep token approval finite and match the planned spender. A webpage, repository instruction or model suggestion cannot broaden those rules.

Before execution, check prerequisites for the selected cases: connectivity, wallet support, chain, balances, allowances, account permissions, token metadata and pool readiness. Intentional negative-test conditions such as an empty wallet are fixtures, not funding blockers.

Recovery rule

The proposed cap is three attempts total: initial execution and two retries. Stop sooner for conclusive evidence, a violated policy, an exhausted budget or an unsafe-to-repeat transaction. Successful recovery after a failure is reported as FLAKY when relevant to the case outcome.

Condition	Runner response
Clear reproducible app mismatch	Record a defect; reproduce only when useful and permitted
Temporary infrastructure or browser issue	Retry within case and job limits
Three attempts with unresolved outcome	Stop case as INCONCLUSIVE or RUNNER_ERROR with supporting history
Missing prerequisite	BLOCKED and a targeted checkpoint
Failed dependency	NOT_RUN with dependency reference

Continue independent cases when their prerequisites hold. Never change assertions to manufacture a pass. A selector repair or wait adjustment must preserve the approved expectation and be logged.

Retries of signed transactions require chain reconciliation first. Retain the submitted hash and nonce; inspect receipt/pending status before another signature. If confirmation is unknown, pause instead of sending another swap. Browser retry counters are not sufficient protection against duplicate onchain effects.

Checkpoints and scope changes

At a checkpoint, show completed work, the blocking evidence, the exact missing input and the remaining budget. Pause billable execution resources while waiting, retaining state needed to resume.

A funded protocol deadline continues to approach during a pause. Configure realistic delivery and review windows; do not imply the UI can freeze or extend escrow deadlines. If the job cannot resume in time, deliver an honest incomplete report and make human review available.

Additional scope or higher spending requires a revised quote and approval. Use a new job where required by the payment lifecycle. Do not repeatedly ask about routine actions already covered by the approved plan.

Every policy check, attempted action, retry, checkpoint, actual usage increment and stop reason belongs in the session event log described in Session evidence and refund review.

Wallet QA agent session evidence and refund review

Each delivered report has a recording and a structured account of the session. These records support a human assessment of what happened, why a case stopped, and whether the service met its agreed obligations. Responsibility remains unknown when the available evidence cannot establish it.

Record every relevant interaction and action

Record intake messages, agent questions, user answers, approval/revision events, funding requests, preflight checks, browser actions, wallet prompts, signature decisions, transactions, model-directed recovery, retries, checkpoints, limits and final results. Store concise decisions and observable actions rather than private model reasoning.

Use a monotonically sequenced append-only event stream with UTC timestamps for correlation. Display times to the user in their timezone. Each event includes session/job IDs, plan version, case/step/attempt IDs where applicable, actor, action, sanitized inputs, observation, result, related transaction reference, usage counters and evidence references.

Capturing every relevant action does not require storing credentials or secrets. Keep provisioning and wallet unlock credentials outside recordings; log that those operations occurred with sensitive values omitted. Protect private documents and redact authentication headers before delivery. Flag recording gaps explicitly.

Evidence bundle

Artifact	Required value
Approved plan	Exact requirements, expected results, limits and buyer approval
Session log	Ordered events, interactions, retries and stop reasons
Recording	Browser journey and wallet interactions for every attempt
Trace and screenshots	Reproduction detail and assertions tied to steps
Chain evidence	Submitted hashes, receipts, network, amounts, allowances and balance changes
Result report	Case outcomes, actual versus expected behavior and unresolved work
Manifest	Artifact IDs, byte sizes, digests and links to cases/events

Align video offsets with event times. Preserve original failure evidence when a retry succeeds. Validate capture across dApp pages and wallet windows; one page recording may omit an extension popup. [Playwright video](#), [Trace viewer](#)

Provide a whole-session replay that pairs conversations and approvals with synchronized browser/wallet recordings. Capture relevant chat and approval screens; retain original asynchronous messages as timestamped replay entries. Show waiting periods and recording gaps explicitly. Later review events remain linked to the original session.

Seal the execution bundle before submitting its result hash. Hash exact delivered bytes using the applicable Masumi input/output hashing rules; follow-ups must not mutate that delivered artifact. Store later review and settlement events in a continuing audit journal referencing the sealed bundle. An amended report is a new version with its own digest.

Hashes detect later changes when compared with a trusted anchor. They do not establish that capture was honest, prove unseen events, or automatically decide fault. Human reviewers may need independent receipt checks, a replay, or additional buyer/operator evidence.

Assess the blocker

For each stopped case, include the approved precondition, observed state, last successful step, attempted recovery, evidence and remaining uncertainty. Keep `case_outcome`, `cause_category`, `attribution_status` and `delivery_complete` separate.

Observed evidence	Proposed interpretation
Required funds absent despite a recorded preflight check	Investigate service validation failure and later wallet activity
Supplied account denied permission	Account/environment blocker; determine whether the requirement was discoverable and confirmed
Submitted swap succeeds but UI notification is absent	App mismatch if notification was an approved assertion
Locator fails although the expected element is visible	Likely runner problem; inspect trace and replay
RPC times out and transaction status is unresolved	External or unknown cause; stop resubmission
Expected behavior was never confirmed	Specification gap; do not invent an app defect
Three attempts without diagnostic evidence	Unresolved outcome; eligible for human delivery review

These interpretations are hypotheses until reviewed. A blocker involving a third party is not automatically outside the operator's responsibility: the service may have promised to handle that dependency or validate it before accepting the job.

Human review policy

The buyer selects affected cases and a reason from the report. The agent assembles the approved terms, evidence and attempts. A human operator makes the seller-side decision and records its rationale.

Delivery condition	Proposed remedy
Reproducible app defect with required proof	Testing delivered; no refund solely because the app failed
Promised critical work not executed because of the service	Human refund or replacement-run review
Required evidence missing or unusable	Human refund or replacement-run review
Inconclusive or disputed attribution	Review agreed scope and evidence; do not automatically blame buyer or service
New requested work	New plan and quote

Start with full-job refunds or replacement runs. Partial escrow refunds remain an integration decision, not an MVP promise. Show the actual review deadline and reminders. A support ticket must be paired with the necessary protocol request before that deadline; after settlement, reimbursement is a separate operator decision.

Masumi protocol boundary

Buyer requests use `/purchase/request-refund` before `unlockTime`; accepted seller requests use `/payment/authorize-refund`. Unresolved disputes escalate to the Masumi team. The app's human reviewer cannot independently replace that escalation authority. Configure delivery, review and escalation timestamps explicitly and read live payment state. [Masumi Refunds and Disputes](#)

How to Avoid Refunds checklist

Adapted from the linked Masumi seller checklist:

- Describe supported scope accurately in the registry.
- Publish a genuine example deliverable.
- Monitor credentials, dependencies and uptime.
- Hash exact input and output bytes consistently.
- Set conservative delivery deadlines with a buffer.

The product applies these through explicit assertions, preflight checks, complete evidence and a named human reviewer. Service operations should detect missing artifacts and approaching deadlines before the buyer has to report them.

Wallet QA agent architecture and implementation sequence

Use a thin product layer over existing wallet automation and Masumi settlement. Keep planning, deterministic execution, evidence storage and payment state separate so failures can be diagnosed and reviewed.

Components

Component	Responsibility
Web application	Intake, structured interview, plan/quote approval, progress, report and human review
Job orchestrator	Persist workflow state, enforce limits, schedule runner, handle pauses and deadlines
Planning service	Extract relevant context and propose concise cases; consume structured deltas
Browser runner	Execute approved scripts through real wallet interactions
Signing policy	Enforce approved network, contracts, amounts, actions and transaction limits
Chain reader	Reconcile receipts, allowances, balances and pending transactions
Evidence store	Recordings, traces, screenshots, event logs and sealed manifests
Masumi adapter	Register service, link jobs/payments, submit hashes, request/authorize refunds and observe settlement
Human operator view	Compare terms/evidence and record refund or replacement decisions

Next.js/TypeScript is a suitable frontend and orchestration default. A persistent Chromium runner with Playwright and Synpress is the proposed first wallet path. A worker/container is needed for long browser jobs; do not run the entire session inside a short HTTP request.

Synpress provides MetaMask automation. Playwright supports extensions in persistent Chromium. Browserbase supports custom extensions, but the exact wallet/runner/capture combination must pass an integration spike before committing to hosted execution. [Synpress](#), [Playwright extensions](#), [Browserbase extensions](#)

State and persistence

Application workflow: draft, inspecting, awaiting clarification, plan ready, awaiting funding, running, paused, report ready, under human review, closed or canceled.

Keep this separate from Masumi service status, payment status and per-case outcome. A ready report does not imply accepted delivery, and a technical job completion does not imply payment collection.

Persist job identity, requester, environment, plan versions, quote, approval, limits, wallet references, transaction references, checkpoint requests, case attempts, artifact manifests and human decisions. Append history rather than overwriting failures with the latest outcome.

Validate client ownership and plan version on mutations. Use idempotency for job creation, funding linkage and transaction actions. Chain events and queued callbacks can arrive late or more than once.

Limit browser access and signing to the approved app environment. Treat page and repository content as data; it cannot authorize secret disclosure, new domains/contracts or larger spending. The backend

enforces those boundaries independently of model output.

Masumi integration

Use the standard service endpoints for job start, status, availability and input schema; implement the optional additional-input endpoint for checkpoints. Bind job IDs to `blockchainIdentifier` and read live payment state before starting paid execution. Quote and plan approval endpoints are application features. [MIP-003](#), [Payments and Escrow](#)

Choose the payment asset and node hosting with the partner team. The standard escrow route is sufficient for the main paid job. x402 is an optional request/payment entry point; direct address payment alone does not provide the same escrow review process. [Masumi x402](#)

Use protocol timestamps and hashing rules from the deployed integration. Application pausing must not be interpreted as an extension of those timestamps.

Implementation sequence

1. Prove one real wallet connection, finite approval and swap on the selected testnet; capture wallet windows.
2. Prove one Masumi preprod lock, result submission and seller-authorized refund. Establish the human reviewer and review deadlines.
3. Define the shared job/test/event records and versioned approval. Build the structured interview and fixed quote.
4. Connect the approved plan to the bounded runner, transaction reconciliation and evidence store.
5. Build the report and human review screen; preserve all attempted outcomes.
6. Complete the demo checks, recording and hackathon submission material.

The first two spikes determine viability. Existing public tooling may be studied now; the submitted product and substantial code must be built within the event under the supplied rules.

Later interfaces

Agent-to-agent mode is paused. Preserve machine-readable schemas and status so external agents can later submit requests, answer essential questions, bring plans back for authorization and retrieve artifacts. Spending delegation, signed plan authority and duplicate paid requests need design before enabling autonomous clients.

Telegram remains optional. Build it as another authenticated client of the same job workflow. Use chat for interview, progress and report delivery; connect funding and detailed approval to an authenticated web or Mini App experience. Store bot tokens on the server when implementation begins.

Wallet QA agent swap demo and acceptance

Demonstrate the complete hiring and testing flow on one small swap app. The product value is agreement on expectations, real wallet execution, linked evidence, and understandable findings.

Demo setup

Proposed environment: MetaMask, Chromium, Base Sepolia, two known ERC20 test tokens, a funded pool and one fixed swap amount. Select the exact app and contracts during the technical spike. Use public reference tooling; avoid designing a new swap protocol.

Use isolated wallets for the main and negative scenarios. The main wallet begins with enough gas and input tokens, and zero allowance for the approved spender. Masumi on Cardano preprod handles the QA job's service payment.

The demonstration can use a deliberately planted missing-success-notification defect. Label it as planted. The transaction must still execute on the real testnet, and the report must reflect actual captured observations.

Main scenario

Step	Action	Assertion and proof
1	Connect wallet	Correct account and network displayed; wallet/browser recording
2	Enter fixed amount	Correct tokens, amount, quote and approved minimum output
3	Approve finite allowance	Intended spender and amount; approval receipt and allowance
4	Submit one swap	Wallet confirmation, submitted hash and successful receipt
5	Check balances	Input/output balance changes and output at least the approved minimum
6	Check notification	Pending and success behavior agrees with confirmed requirements and timeout

Receipt success and UI success are distinct assertions. If the chain succeeds but the required notification is absent, report the UI defect without resubmitting the swap.

Small negative scenario

Reject the token approval. Verify that no approval transaction is submitted, allowance remains unchanged, the app leaves its loading state and another action remains possible. Keep this scenario independent from the successful swap fixture.

Product demonstration sequence

1. Submit the app URL and swap goal.
2. Show discovered context and a few essential questions.
3. Confirm a concise test plan, price, funds and limits.
4. Show real service escrow funding.
5. Run the swap and cancellation scenarios with recordings and step events.

6. Open the report, compare UI and chain results, and play a relevant evidence segment.
7. Show the human-review route for incomplete service delivery using a separately labeled controlled runner-failure job if time permits. An app defect alone is not grounds for a refund.

Delivery acceptance checks

- The approved plan links each case to a requirement and explicit expectations.
- The quote shows included contingency, recovery limits, testing funds and actual review deadline.
- Service funds lock before paid execution.
- Real wallet connection, bounded approval and swap occur on the chosen testnet.
- A pending or successful transaction is not duplicated during recovery.
- Each attempted action, interaction and retry appears in the session log.
- The recording includes relevant wallet windows and browser behavior; capture gaps are disclosed.
- The session replay also exposes conversations, approvals and checkpoints with their original timestamps.
- Evidence references resolve, digests match delivered bytes, and report results match the observed records.
- The system stops unresolved recovery after its approved attempt/time limit; dependent work is marked NOT_RUN.
- Failed and flaky attempts remain visible; missing proof never becomes PASS.
- A human can request/review delivery issues, with confirmed protocol request state before the deadline.
- A scoped report, recording, trace and event log can be downloaded by the buyer.

Do not claim production readiness, all-site coverage or automatic certainty about fault from this demo.

Submission recording

Prepare a short recording of the product journey for the stage deck, separate from the complete buyer evidence recording. Embed the video directly in the required PPT or Keynote file. The supplied brief prohibits live final-stage demos and external video links; submit the deck, repository and hosted demo before the stated deadline.